COMPUTER SCIENCE

O Level Notes

Syllabus 2210 / 0478

Prepared by

Khaled Bin Aziz

01756922708

© 2026 · All Rights Reserved

O Level Computer Science

Chapter 2: Data Transmission

2.1 Types and Methods of Data Transmission · 2.2 Error Detection · 2.3 Encryption

---

1. Data Packets

Data sent over a network (especially over long distances) is split into small units called **data packets**. Each packet is usually about **64 KiB**.

Why use packets?
- Small packets are easier to control than one long stream of data.
- Each packet can travel along a **different route** to the destination.
- If a route is busy or broken, packets can use another route.

Drawback: the packets must be **reassembled** in the correct order at the destination.

1.1. Packet Structure

Every packet has **three parts**:

Part	What it contains
Header	• **IP address of the sender** • **IP address of the receiver** • **Sequence (packet) number**: used to put the packets back in the correct order • **Packet size**: used to check that the whole packet arrived
Payload	The **actual data** being sent (about 64 KiB)
Trailer	• Shows the **end of the packet** • **Error checking** (cyclic redundancy check, CRC)

> **Exam Tip:** Header = who + order + size. Payload = data. Trailer = end marker + error check.

1.2. Cyclic Redundancy Check (CRC)

The CRC is the error check stored in the **trailer**.

1. The sender **adds up all the 1-bits** in the payload.
2. This total is stored as a **hex value** in the trailer.
3. The packet is sent.
4. The receiver **recalculates** the number of 1-bits in the payload.
5. The receiver **compares** its value with the value in the trailer.
6. If the values **match**, there is no error. If they **differ**, the packet must be **re-sent**.

KHALED BIN AZIZ

2. Packet Switching

Packet switching is a method of transmission in which a message is broken into packets, and each packet is sent **independently** (often by a different route) from the sender to the receiver.

How packet switching works:

1. The message (e.g. a photograph) is **split into packets**.
2. Each packet is given a **header** containing the destination IP address and a sequence number.
3. Each packet is sent to a **router**.
4. The router reads the **destination IP address** in the header and decides the **best next route** for that packet.
5. Each packet may take a **different route**, so packets may arrive **out of order**.
6. The receiving computer **reassembles** the packets in the correct order using the **sequence numbers**.

> **Router:** a device (at each node of the network) that receives a packet and decides where to send it next, using the information in the header. The route chosen depends on how busy each path is, so it is not always the shortest path.

Benefits	Drawbacks
No need to tie up a **single communication line**	Packets can be **lost** and must be re-sent
Busy, faulty or failed lines are **avoided** by re-routing	**Delay** at the destination while packets are re-ordered
Easy to **expand** network usage	More prone to errors with **real-time streaming** (e.g. a live sports event)
High transmission rate is possible	

3. Direction of Data Transmission

Mode	Meaning	Example
Simplex	Data goes in **one direction only**	Computer ⟶ printer
Half-duplex	Data goes in **both directions**, but **not at the same time**	Walkie-talkie
Full-duplex	Data goes in **both directions at the same time**	Broadband internet, telephone call

> **Exam Tip:** Simplex = **one way**. Half-duplex = **two ways, one at a time**. Full-duplex = **two ways, together**.

4. Serial and Parallel Transmission

Serial: data is sent **one bit at a time** down a **single wire (channel)**.

Parallel: several bits (usually one byte) are sent **at the same time** down **several wires**. Each wire carries one bit.

Serial	Parallel
One wire, **one bit at a time**	Several wires, **several bits at a time**
Slower	**Faster**
Reliable over long distances	Works well only over **short distances**
Bits stay in order, so **no skew**	**Skew** can happen: bits arrive out of order
Fewer wires: less interference and **cheaper**	More wires: more interference and **more expensive**
Example: USB, telephone lines	Example: inside the computer (between CPU and memory, circuit boards)

> **Data skew:** the bits arrive at the destination **out of synchronisation** (not together or not in order), which can corrupt the data. It happens in **parallel** transmission over long wires. The longer the wire, the worse it gets.

> **Exam Tip:** Long distance → **serial**. Short distance and speed is important → **parallel**. Serial and parallel can each be simplex, half-duplex or full-duplex.

5. Universal Serial Bus (USB)

USB is a type of **serial** data transmission. It is the **industry standard** for connecting devices to a computer. USB supports **half-duplex and full-duplex** transmission.

USB cable: 4 wires in a shielded cable.
- **2 wires for power** (red +5V and black ground)
- **2 wires for data** (white and green)

What happens when a USB device is plugged in?
1. The computer **automatically detects** the device.
2. The device is recognised and the correct **device driver** is loaded automatically.
3. If the driver is not available, the user is asked to download it.

Benefits	Drawbacks
Devices are **detected automatically** and drivers are loaded	Maximum cable length is about **5 m** (hubs are needed for longer)
Connector fits **only one way**, so wrong connections are prevented	Data transfer rate is **slower than Ethernet**
Industry standard, so lots of support is available	Very old USB versions may **not be supported** by new computers
Cable supplies **+5V power**, so no external power is needed	
Errors are detected and the data is **re-transmitted**	
Easy to add more ports using **USB hubs**	
Backward compatible with older versions	

> **USB-C:** a small, thin, 24-pin connector that fits **either way round**. It carries data at about 10 Gbps, supports 4K video and can deliver 100 W of power for charging.

6. Why Check for Errors?

When data is transmitted it can be **corrupted, lost or even gained**.

Causes of errors:
- **Interference** on the cable (data is corrupted or lost)
- **Problems during packet switching** (packets are lost)
- **Skewing** during parallel transmission (bits arrive out of order)

A computer cannot make sense of corrupted data, so error checking is essential.

Methods that check data after transmission: parity check, checksum, echo check and ARQ.

Method that checks data entry: check digit.

7. Parity Check

A **parity bit** is an extra bit added to a byte. It is usually the **left-most bit** (most significant bit).

- **Even parity:** the byte must contain an **even** number of 1-bits.
- **Odd parity:** the byte must contain an **odd** number of 1-bits.

The sender and receiver **agree** on the type of parity **before** the data is sent.

Example: the 7-bit data `1110000` has three 1-bits.
- Even parity: the parity bit is `1`, so the byte is `11110000`.
- Odd parity: the parity bit is `0`, so the byte is `01110000`.

7-bit data	Number of 1s	Even parity bit	Odd parity bit
`1010101`	4	0	1
`1011000`	3	1	0
`0001000`	1	1	0

How a parity check works:
1. Sender and receiver agree on even or odd parity.
2. The sender adds the parity bit to each byte and sends the data.
3. The receiver counts the 1-bits in each byte received.
4. If the parity is **wrong**, an error has been detected and the data is **re-sent**.

> **Limitation:** if **two bits** change, the parity can still look correct, so the error is **not detected**. A single parity bit also does not show **which** bit is wrong.

7.1. Parity Block (Parity Byte)

Data is sent as a **block of bytes**, followed by an extra **parity byte**. The parity byte is made from the **vertical (column) parity bits**.

- Each **row** (byte) is checked for correct parity.
- Each **column** is checked for correct parity.

KHALED BIN AZIZ

- The wrong bit is where the **incorrect row** and the **incorrect column** meet.
- Flip that bit (0 ↔ 1) to **correct** the error, or ask for the block to be re-sent.

Example (even parity). One bit was changed during transmission (shaded):

	Parity	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	1	0	1	1	0	1	0	0
Byte 2	0	1	1	0	0	1	1	0
Byte 3	1	1	0	1	**1**	0	0	1
Byte 4	1	0	1	0	1	1	0	0
Parity byte	1	0	1	0	1	1	1	1

- **Byte 3** has five 1-bits (odd), so the **row** is wrong.
- **Column 5** has three 1-bits (odd), so the **column** is wrong.
- The error is at **Byte 3, Bit 5**. It should be `0`, so the correct byte is `11010001`.

> **Exam Tip:** A parity block can **find and correct** the wrong bit. A single parity bit can only **detect** that an error exists.

8. Checksum

A **checksum** is a value calculated from a **block of data** and sent **after the block**.

1. The sender calculates the checksum using an **agreed algorithm** and sends it with the block of data.
2. The receiver **recalculates** the checksum from the data it received, using the same algorithm.
3. The receiver **compares** the two checksums.
4. If they are the **same**, there is no error. If they are **different**, the block of data is **re-sent**.

9. Echo Check

1. The data is sent to the receiver.
2. The receiver **sends a copy of the data back** to the sender.
3. The sender **compares** the returned data with the original data.
4. If they are the **same**, there was no error. If they **differ**, an error occurred and the data is re-sent.

> **Weakness:** if the data is different, we do not know whether the error happened on the way **to** the receiver or on the way **back**. So echo check is **not very reliable**.

10. Automatic Repeat Request (ARQ)

ARQ uses **acknowledgements** and a **timeout**.

- **Positive acknowledgement:** a message to say the data arrived **correctly**.
- **Negative acknowledgement:** a message to say the data had an **error** and must be re-sent.
- **Timeout:** the time the sender waits for an acknowledgement.

How ARQ works:
1. The sender sends the data.

2. The receiver checks the data for errors (usually using a **CRC**).
3. If there is **no error**, the receiver sends a **positive acknowledgement**.
4. If there is an **error**, the receiver sends a **negative acknowledgement** and asks for the data to be re-sent.
5. The sender waits for the **timeout** period. If **no acknowledgement** arrives in that time, it **re-sends automatically**.
6. This repeats until a positive acknowledgement is received **or** a set number of re-sends has been made.

> **Exam Tip:** ARQ = **acknowledgement + timeout**. It is often used by mobile phone networks.

11. Check Digits

A **check digit** is the **last digit** of a code. It is **calculated from all the other digits** and is used to detect **data entry errors** (typing or scanning mistakes).

Used in: barcodes, ISBNs (book numbers), Vehicle Identification Numbers (VIN).

> A check digit checks **data entry**. It is **not** a data transmission check.

Errors a check digit can detect:

Type of error	Correct code	Entered code
Incorrect digit	5307	5327
Transposition (two digits swapped)	5307	5037
Omitted digit	5307	537
Extra digit	5307	53107
Phonetic error	30 (thirty)	13 (thirteen)

11.1. ISBN-13 Check Digit

Finding the check digit from the first 12 digits:

1. Add the **odd-position** digits (1st, 3rd, 5th, …).
2. Add the **even-position** digits (2nd, 4th, …) and **multiply by 3**.
3. Add the two results and **divide by 10**.
4. If the remainder is 0, the check digit is 0. Otherwise **check digit = 10 − remainder**.

Example: `978034098382`

```
Odd digits : 9 + 8 + 3 + 0 + 8 + 8       = 36
Even digits: 3 × (7 + 0 + 4 + 9 + 3 + 2) = 3 × 25 = 75
Total      : 36 + 75 = 111
111 ÷ 10   = 11 remainder 1
Check digit = 10 − 1 = 9
```

Full ISBN-13: `9780340983829`

Checking a 13-digit number: include the check digit in the odd-position sum.

```
Odd digits : 9 + 8 + 3 + 0 + 8 + 8 + 9 = 45
Even digits: 3 × 25 = 75
Total      : 45 + 75 = 120
120 ÷ 10   = 12 remainder 0   →   number is CORRECT
```

If the remainder is **0**, the number is correct.

11.2. Modulo-11 Check Digit

Finding the check digit (example with a 7-digit number):

1. Give each digit a **weighting**: 8, 7, 6, 5, 4, 3, 2 (from the left).
2. **Multiply** each digit by its weighting and **add** the results.
3. **Divide the total by 11** and find the **remainder**.
4. **Check digit = 11 − remainder.** (If the answer is 10, the check digit is **X**.)

Example: `4156710`

```
(8×4) + (7×1) + (6×5) + (5×6) + (4×7) + (3×1) + (2×0)
= 32 + 7 + 30 + 30 + 28 + 3 + 0 = 130
130 ÷ 11 = 11 remainder 9
Check digit = 11 − 9 = 2      →   41567102
```

Checking an 8-digit number: the check digit gets a weighting of **1**.

```
(8×4) + (7×1) + (6×5) + (5×6) + (4×7) + (3×1) + (2×0) + (1×2) = 132
132 ÷ 11 = 12 remainder 0     →   number is CORRECT
```

If the remainder is **0**, the number is correct.

11.3. Checksum vs Check Digit

Checksum	Check Digit
Checks **data transmission** errors	Checks **data entry** errors
Calculated from a **block of data**	Calculated from **one number/code**
Sent **after the block** of data	Is the **last digit** of the code
Calculated by the **computers** (sender and receiver)	Checked when a **person types** or a scanner reads the code
If the values differ, the **block is re-sent**	If the values differ, the **person re-enters** the code

11.4. Summary of Error Detection Methods

Method	Key idea
Parity check	An **extra bit** is added to each byte to make the number of 1s even or odd
Parity block	Parity checked on **rows and columns**; finds the exact wrong bit
Checksum	A **calculated value** is sent after a block; the receiver recalculates and compares
Echo check	Data is **sent back** to the sender and compared
ARQ	**Acknowledgement + timeout**; data is re-sent if there is an error or no reply
Check digit	**Last digit** of a code; detects **data entry** errors

12. Purpose of Encryption

Data sent over a **public network** (wired or wireless) can be **intercepted** by a hacker. A hacker who intercepts data is called an **eavesdropper**.

Encryption changes data into a form that **cannot be understood** by anyone except the intended receiver.

- Encryption does **not stop** data being intercepted.

- It makes the intercepted data **meaningless** to the eavesdropper.
- It protects **sensitive data**, such as bank details, medical records and passwords.

Term	Meaning
Plaintext	The **original data** before encryption
Ciphertext	The data **after** it has gone through the encryption algorithm
Encryption algorithm	Converts plaintext into ciphertext using a key
Key	The value used to encrypt and decrypt the data

```
Plaintext → Encryption algorithm + key → Ciphertext → Decryption + key → Plaintext
```

13. Symmetric Encryption

Symmetric encryption uses the **same key** to **encrypt and decrypt** the message.

- Modern keys are long (for example **256-bit**), which makes them very hard to crack.
- **Drawback (key distribution problem):** the key must be **sent to the receiver**, and it could be **intercepted**. Anyone who gets the key can decrypt the data.

> **Exam Tip:** Main weakness of symmetric encryption = **the key must be shared, so it can be intercepted**.

14. Asymmetric Encryption

Asymmetric encryption uses **two keys** that are a **matching pair**:

Key	Who has it?	Used to
Public key	Available to **everybody**	**Encrypt**
Private key	Known **only to the owner**	**Decrypt**

Example: Tom wants to send a confidential file to Jane.

1. Jane generates a **matching pair** of keys (public and private).
2. Jane sends her **public key** to Tom.
3. Tom **encrypts** the file using Jane's **public key** and sends the ciphertext to Jane.
4. Jane **decrypts** it using her **private key**.

> - The public key **cannot decrypt** the message. Only the **matching private key** can.
> - The private key is **never sent** to anyone.
> - For Jane to send encrypted files **back** to Tom, Tom must also make a key pair and send Jane his public key.

Why is asymmetric encryption more secure than symmetric?

- The **private key is never shared**, so it cannot be intercepted.
- There is **no key distribution problem**.

Symmetric	Asymmetric
One key for encryption and decryption	**Two keys**: public and private

The key must be **shared**	Only the **public key** is shared
Less secure	**More secure**

15. Model Answers

Describe how an echo check works.
- The data is sent to the receiver, who sends a copy back to the sender.
- The sender compares the returned data with the original data.
- If they are different, an error occurred.

Why might echo check be unsuitable?
- If the data is different, we do not know whether the error happened on the way there or on the way back.

Describe how ARQ works.
- The receiver checks the data. If it is correct, it sends a positive acknowledgement.
- If there is an error, it sends a negative acknowledgement and the data is re-sent.
- If no acknowledgement arrives before the timeout, the sender re-sends the data automatically.
- This repeats until a positive acknowledgement is received or a set number of re-sends is reached.

Give three differences between checksum and check digit.
- Checksum checks transmission errors; a check digit checks data entry errors.
- Checksum is calculated from a block of data; a check digit from a single code.
- Checksum is sent after the block; a check digit is the last digit of the code.

Explain what is meant by packet switching.
- The message is split into packets, and each packet is sent independently by a route chosen by the routers.
- The packets are reassembled in the correct order at the destination.

Explain data skew.
- Bits sent in parallel arrive at the destination out of synchronisation.

Explain why encryption is used.
- Data sent over a network can be intercepted by a hacker.
- Encryption makes the data meaningless to anyone who does not have the key.

16. Common Exam Questions

- Describe the structure of a data packet (header, payload, trailer).
- Describe how packet switching works and the role of the router.
- State benefits and drawbacks of packet switching.
- Explain simplex, half-duplex and full-duplex with examples.
- Compare serial and parallel transmission, and explain data skew.
- State benefits and drawbacks of USB.
- Find the parity bit for a byte (odd or even), or find the incorrect bit in a parity block.
- Describe how checksum, echo check and ARQ work.
- Calculate or verify a check digit (ISBN-13 or modulo-11).
- State the differences between checksum and check digit.
- Explain why encryption is needed.
- Compare symmetric and asymmetric encryption.

KHALED BIN AZIZ

17. Quick Revision

- A packet has a **header** (sender IP, receiver IP, sequence number, size), a **payload** (data) and a **trailer** (end marker + CRC).
- Packet switching: packets take **different routes**, **routers** choose the route using the destination IP, and packets are **reassembled** using the sequence numbers.
- Simplex = one way. Half-duplex = both ways, not together. Full-duplex = both ways together.
- Serial = 1 bit at a time, good for long distances. Parallel = several bits at once, faster but can **skew**.
- USB is **serial**, has **4 wires**, a maximum cable length of about **5 m**, and is detected automatically.
- Errors are caused by **interference, packet switching problems and skewing**.
- **Parity:** an extra bit makes the number of 1s even or odd. **Checksum:** a calculated value sent after a block.
- **Echo check:** data is sent back and compared. **ARQ:** acknowledgement + timeout.
- **Check digit:** the last digit, detects **data entry** errors (ISBN-13 divides by 10, modulo-11 divides by 11).
- **Encryption** turns **plaintext** into **ciphertext** to protect data from eavesdroppers.
- **Symmetric** = same key (key can be intercepted). **Asymmetric** = public key encrypts, private key decrypts (more secure).

KHALED BIN